

Problem Statement: Breast Cancer Diagnosis Using Logistic Regression**Objective:**

Predict whether a tumor is malignant or benign using patient data features.

Evaluate model performance using accuracy, confusion matrix, precision, and recall.

Visualize feature importance and confusion matrix to interpret model predictions.

Dataset:

Sklearn's Breast Cancer dataset

569 samples, 30 features (e.g., mean radius, mean texture)

Target variable: 0 → Malignant, 1 → Benign

Steps to Solve the Problem**Step 1: Import Libraries**

Import necessary libraries for data manipulation (pandas, numpy), model building (sklearn), and visualization (matplotlib, seaborn).

Step 2: Load Dataset

Load the Breast Cancer dataset from sklearn.

Separate features (X) and target (y).

Step 3: Train-Test Split

Split data into training (80%) and testing (20%) sets.

Ensures model is evaluated on unseen data.

Step 4: Feature Scaling

Scale features using StandardScaler to normalize the range.

Helps Logistic Regression converge faster and treat all features equally.

Step 5: Train Logistic Regression Model

Create a Logistic Regression model and train it on the scaled training data.

Model learns weights for each feature to separate malignant and benign classes.

Step 6: Make Predictions

Predict class labels (0 or 1) for the test data.

Optionally, calculate predicted probabilities for each class.

Step 7: Evaluate Model

Calculate Accuracy, Precision, Recall, and Confusion Matrix.

Understand false positives (predicted benign but actually malignant) and false negatives (predicted malignant but actually benign).

Step 8: Visualize Results

Confusion Matrix Heatmap – visual representation of TP, TN, FP, FN.

Step 0: Import Libraries

```
import pandas as pd
import numpy as np

# Dataset
from sklearn.datasets import load_breast_cancer

# Train-test split
from sklearn.model_selection import train_test_split

# Logistic Regression
from sklearn.linear_model import LogisticRegression

# Feature scaling
from sklearn.preprocessing import StandardScaler

# Evaluation metrics
from sklearn.metrics import accuracy_score, confusion_matrix, precision_score, recall_score
```

Step 1: Load Dataset

```
# Load Breast Cancer dataset
data = load_breast_cancer()

# Features and target
X = pd.DataFrame(data.data, columns=data.feature_names)
y = pd.Series(data.target) # 0 = malignant, 1 = benign

# Quick look at data
print(X.head())
print(y.value_counts())
```

```
   mean radius  mean texture  mean perimeter  mean area  mean smoothness  \
0      17.99      10.38      122.80      1001.0      0.11840
1      20.57      17.77      132.90      1326.0      0.08474
2      19.69      21.25      130.00      1203.0      0.10960
3      11.42      20.38       77.58       386.1      0.14250
4      20.29      14.34      135.10      1297.0      0.10030

   mean compactness  mean concavity  mean concave points  mean symmetry  \
0      0.27760      0.3001      0.14710      0.2419
1      0.07864      0.0869      0.07017      0.1812
2      0.15990      0.1974      0.12790      0.2069
3      0.28390      0.2414      0.10520      0.2597
4      0.13280      0.1980      0.10430      0.1809

   mean fractal dimension  ...  worst radius  worst texture  worst perimeter  \
0      0.07871  ...      25.38      17.33      184.60
1      0.05667  ...      24.99      23.41      158.80
2      0.05999  ...      23.57      25.53      152.50
3      0.09744  ...      14.91      26.50       98.87
4      0.05883  ...      22.54      16.67      152.20

   worst area  worst smoothness  worst compactness  worst concavity  \
0      2019.0      0.1622      0.6656      0.7119
1      1956.0      0.1238      0.1866      0.2416
2      1709.0      0.1444      0.4245      0.4504
3       567.7      0.2098      0.8663      0.6869
4      1575.0      0.1374      0.2050      0.4000

   worst concave points  worst symmetry  worst fractal dimension
0      0.2654      0.4601      0.11890
1      0.1860      0.2750      0.08902
2      0.2430      0.3613      0.08758
3      0.2575      0.6638      0.17300
4      0.1625      0.2364      0.07678

[5 rows x 30 columns]
1    357
0    212
Name: count, dtype: int64
```

Step 2: Train-Test Split

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

Step 3: Feature Scaling

```

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

```

Step 4: Train Logistic Regression

```

model = LogisticRegression(max_iter=10000)
model.fit(X_train_scaled, y_train)

```

```

▼ LogisticRegression ⓘ ?
LogisticRegression(max_iter=10000)

```

Step 5: Make Predictions

```
y_pred = model.predict(X_test_scaled)
```

Step 6: Evaluate Model

```

accuracy = accuracy_score(y_test, y_pred)
cm = confusion_matrix(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)

print("Accuracy:", accuracy)
print("Confusion Matrix:\n", cm)
print("Precision:", precision)
print("Recall:", recall)

```

```

Accuracy: 0.9736842105263158
Confusion Matrix:
[[41  2]
 [ 1 70]]
Precision: 0.9722222222222222
Recall: 0.9859154929577465

```

Classification with Visualizations

```

# Libraries for visualization
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(6,4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=['Malignant','Benign'], yticklabels=['Malignant','Benign'])
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix')
plt.show()

```



